

Pharma Search – An Intelligent Medicine Search and Recommendation System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING

by

Monish Putti : 2520030395

Hemasai : 2520030599

Nikhil Reddy : 2520030560

Under the Guidance of

Dr. V.Sireesha

Professor

**Department of Computer Science and
Engineering**



2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “**Pharma Search – An Intelligent Medicine Search and Recommendation System**” is a record of Bonafide work of **Monish Putti – 2520030395, Hemasai – 2520030599, Nikhil Reddy – 2520030560** submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other department/University/Institute.

Monish Putti : 2520030395

Hemasai : 2520030599

Nikhil Reddy : 2520030560

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “**Pharma Search – An Intelligent Medicine Search and Recommendation System**” is a bonafide work done and submitted by **Monish Putti – 2520030395, Hemasai – 2520030599, Nikhil Reddy – 2520030560** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026–27**.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **Dr. V. Sireesha, Professor, Department of Computer Science and Engineering**, for her valuable guidance, continuous support, feedback, and encouragement throughout the development of Pharma Search. Her suggestions helped us improve the system design, algorithmic implementation, documentation, and presentation of the project.

We also express our sincere thanks to **Dr. P. Venkateswara Rao, Associate Professor and HOD, Department of Computer Science and Engineering**, for providing the academic environment and support required to complete this project. We are grateful to the department and university for providing the resources and opportunities necessary for successfully carrying out this work.

Monish Putti : 2520030395

Hemasai : 2520030599

Nikhil Reddy : 2520030560

Abstract

Pharma Search – An Intelligent Medicine Search and Recommendation System is a web-based medicine information search application designed to provide fast and relevant results from a large medicine corpus. The system processes **242,381 medicine records** and supports search across medicine name, composition, manufacturer, type, and pack size.

The search engine combines **Knuth-Morris-Pratt (KMP), Rabin-Karp, Levenshtein Edit Distance, and Cosine Similarity**. KMP provides the main exact-pattern score, Rabin-Karp provides additional rolling-hash pattern detection, Edit Distance supports spelling-tolerant matching, and Cosine Similarity measures textual similarity across medicine fields. The final ranking uses **40% KMP, 30% fuzzy matching, and 30% cosine similarity**.

The project is implemented using **Java 17, Spring Boot, HTML, CSS, JavaScript, and file-based CSV/TXT storage**. The frontend provides search, filters, sorting, favorites, and medicine details, while the backend exposes REST APIs. Additional DSA-3 implementations demonstrate the **Z-Algorithm, optimized dynamic programming, Edmonds-Karp maximum flow, greedy approximation, randomized sampling, and parallel prefix sum** concepts.

Table of Contents

Acknowledgement

Abstract

Table of Contents

1 Introduction

1.1 Overview

1.2 Problem Statement

1.3 Scope of the Project

1.4 Objectives

2 System Requirements

2.1 Functional Requirements

2.2 Non-Functional Requirements

2.3 Software and Hardware Requirements

2.4 Dataset Requirements

3 System Design

3.1 High-Level Architecture

3.2 Data Flow of Pharma Search

3.3 Module Mapping

3.4 Algorithm-to-Feature Mapping

4 Implementation

4.1 Module-Wise Implementation

4.1.1 Dataset Loading and Repository

4.1.2 KMP Pattern Matching

4.1.3 Rabin-Karp Pattern Matching

4.1.4 Edit Distance

4.1.5 Cosine Similarity

4.1.6 Search Ranking

4.2 Front-End Logic and UI Rendering

4.3 API Integration

4.4 DSA-3 CO Implementations

4.5 Algorithm Pseudocode

5 Results

5.1 Test Environment

5.2 Functional Results

5.3 Search Results and User Interface

5.4 Algorithm Demonstration

6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development
References	Books, tutorials, APIs, documentation sites used	

List of Figures

3.1 High-Level Architecture of Pharma Search

1. INTRODUCTION

1.1 Overview

The Pharma Search – An Intelligent Medicine Search and Recommendation System is a Java-based web application developed to provide efficient searching and retrieval of medicine information from a large dataset. The system allows users to search medicines using medicine name, composition, manufacturer, type, and pack size.

The project uses a cleaned medicine dataset containing 242,381 medicine records. The backend is implemented using Java and Spring Boot, while HTML, CSS, and JavaScript are used to provide a browser-based user interface.

The system applies Data Structures and Algorithms concepts to improve search accuracy and relevance. The implemented search pipeline combines KMP string matching, Rabin-Karp pattern matching, Edit Distance, and Cosine Similarity to identify and rank relevant medicine records. An additional Z-Algorithm implementation is included as part of the string-algorithm component.

The system also provides medicine-type filtering, result sorting, favourite medicine functionality, and a relevance-based presentation of search results. The purpose of the project is to demonstrate how algorithmic techniques can be integrated into a practical medicine-search application.

1.2 Problem Statement

Searching for medicine information in a large dataset can become inefficient when users enter incomplete names, spelling variations, compositions, or other medicine-related information.

A simple exact text search may fail when the user's query does not exactly match the stored medicine record. For example, a user may enter an incorrectly spelled medicine name or search using a composition instead of the medicine name.

The problem addressed by this project is to develop an intelligent search system that can process a large medicine dataset and retrieve relevant results using efficient string matching and similarity algorithms.

The system should:

- Search medicines using multiple attributes.
- Handle spelling variations in medicine names.
- Detect exact and approximate text matches.
- Rank search results according to relevance.
- Support medicine-type filtering.

- Allow users to sort search results.
 - Provide a simple and interactive web interface.
 - Demonstrate practical applications of advanced Data Structures and Algorithms.
-

1.3 Scope of the Project

The scope of the Pharma Search system includes the development of a web-based medicine search application capable of processing a large medicine dataset.

The system includes:

- CSV-based medicine dataset storage.
 - Data cleaning and preprocessing.
 - Java-based backend processing.
 - Spring Boot REST API.
 - Browser-based frontend.
 - Medicine search by name, composition, manufacturer, type, and pack size.
 - KMP-based pattern matching.
 - Rabin-Karp rolling-hash pattern matching.
 - Z-Algorithm implementation.
 - Edit Distance for approximate matching.
 - Cosine Similarity for text similarity.
 - Combined relevance scoring and ranking.
 - Medicine type filtering.
 - Result sorting.
 - Favourite medicine functionality.
 - Additional DSA implementations corresponding to CO1–CO6.
-

1.4 Objectives

The main objectives of the project are:

- To develop an intelligent medicine-search system using Java.
 - To process and search a large medicine dataset efficiently.
 - To implement KMP for efficient pattern matching.
 - To implement Rabin-Karp using rolling hashing.
 - To implement the Z-Algorithm for linear-time string matching.
 - To use Edit Distance for handling spelling variations.
 - To apply Cosine Similarity for measuring textual similarity.
 - To combine multiple algorithmic scores to rank search results.
 - To provide filtering and sorting functionality.
 - To develop a user-friendly browser-based interface.
 - To demonstrate the practical application of DSA-3 concepts in a real-world search problem.
-

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The system should:

- Load medicine records from the processed CSV dataset.
- Search medicines using a user-provided query.
- Search across medicine name, compositions, manufacturer, type, and pack size.
- Perform exact pattern matching using KMP.
- Perform hash-based pattern detection using Rabin-Karp.
- Calculate approximate matching using Edit Distance.
- Calculate textual similarity using Cosine Similarity.
- Combine algorithmic scores to rank search results.
- Return relevant medicine results through the REST API.
- Display search results through the frontend.
- Filter medicines by type such as Allopathy, Ayurveda, and Homeopathy.

- Sort results according to relevance, medicine name, and price.
 - Allow users to add medicines to favourites.
-

2.2 Non-Functional Requirements

- Performance: The system should process a large medicine dataset efficiently.
 - Accuracy: Search results should provide relevant medicine records for exact and approximate queries.
 - Scalability: The application should be capable of processing hundreds of thousands of medicine records.
 - Usability: The frontend should provide a simple interface for searching and viewing medicines.
 - Maintainability: Algorithm implementations, backend services, models, repositories, and frontend components should remain logically separated.
 - Reliability: Invalid or incomplete records should not prevent the complete dataset from being loaded.
 - Responsiveness: Search requests should return results without unnecessary delay.
-

2.3 Software and Hardware Requirements

The project requires:

- Java Development Kit 17 or newer.
- Spring Boot 3.5.5.
- Apache Maven.
- Visual Studio Code or another Java-compatible IDE.
- Modern web browser.
- Git and GitHub for version control.
- HTML, CSS, and JavaScript environment for the frontend.
- Sufficient storage for the medicine dataset.

The backend runs using Spring Boot and provides REST API endpoints through port 8080 during local development.

2.4 Dataset Requirements

The project uses a cleaned medicine dataset stored as:

data/processed/medicines_clean.csv

The dataset contains approximately 242,381 medicine records.

The medicine records contain attributes including:

- ID
- Medicine Name
- Price
- Manufacturer
- Type
- Pack Size
- Composition 1
- Composition 2

The dataset is loaded by the Java backend and converted into medicine objects for search processing.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The architecture of Pharma Search consists of the following major components:

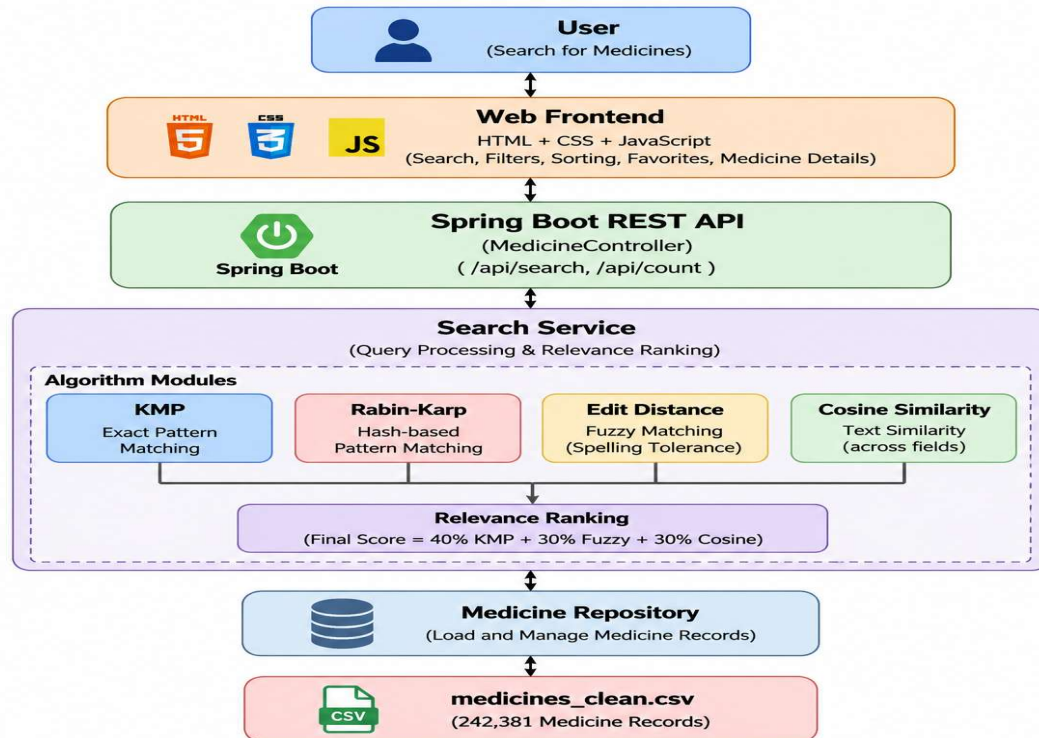


Fig. 3.1 High-Level Architecture of Pharma Search

The frontend sends a search request to the Spring Boot backend. The controller receives the request and passes the query to the Search Service.

The Search Service processes the query using the implemented algorithms and calculates relevance scores for the available medicine records.

3.2 Data Flow

The search process follows these steps:

1. The user enters a medicine-related query.
2. The frontend sends the query to `/api/search`.
3. The Medicine Controller receives the request.
4. The Search Service processes the query.
5. KMP checks for exact pattern matches.
6. Rabin-Karp performs additional hash-based pattern detection.
7. Edit Distance calculates approximate matching.
8. Cosine Similarity measures textual similarity.
9. The scores are combined to calculate the final relevance score.

10. Results are sorted according to relevance.
 11. The top relevant medicines are returned to the frontend.
 12. The frontend displays the results to the user.
-

3.3 Module Mapping

The system contains the following major modules:

- Dataset Module – loads and processes medicine records.
 - Repository Module – stores medicine objects in memory.
 - Search Module – processes user queries.
 - String Algorithm Module – implements KMP, Rabin-Karp, and Z-Algorithm.
 - Fuzzy Matching Module – implements Edit Distance.
 - Similarity Module – implements Cosine Similarity.
 - Ranking Module – combines algorithmic scores.
 - Controller Module – exposes REST API endpoints.
 - Frontend Module – provides the browser interface.
 - Filter and Sort Module – handles medicine type and result ordering.
-

3.4 Algorithm-to-Feature Mapping

Algorithm	Application
KMP	Exact pattern matching
Rabin-Karp	Hash-based pattern detection
Z-Algorithm	String pattern matching
Edit Distance	Spelling variation and fuzzy matching
Cosine Similarity	Text similarity
Combined Ranking	Relevance-based result ordering
Edmonds-Karp	CO4 allocation demonstration

Algorithm	Application
Approximation	CO5 budget recommendation demonstration
Randomized Sampling	CO6 demonstration
Parallel Prefix Sum	CO6 parallel algorithm demonstration

For the main medicine-search pipeline, KMP, Rabin-Karp, Edit Distance, and Cosine Similarity are directly used by SearchService. The additional CO4–CO6 implementations provide separate demonstrations of the corresponding DSA concepts.

4. IMPLEMENTATION

4.1 Module-Wise Implementation

Medicine Dataset

The processed medicine dataset is stored in CSV format and contains approximately 242,381 records.

The MedicineRepository reads the CSV file, parses each record, creates a Medicine object, and stores the records for searching.

Search Service

The SearchService receives the user's query and evaluates it against multiple medicine attributes.

The system checks:

- Medicine name
- Composition 1
- Composition 2
- Manufacturer
- Pack size
- Medicine type

The final relevance score is calculated using:

Final Score =

40% KMP Score

+ 30% Edit Distance Score

+ 30% Cosine Similarity Score

Rabin-Karp is additionally used to detect exact pattern matches.

4.2 Front-End Logic and UI Rendering

The frontend is implemented using HTML, CSS, and JavaScript.

The interface provides:

- Medicine search bar.
- Search button.
- Example search queries.
- Medicine result cards.
- Medicine type filters.
- Sorting options.
- Favourite functionality.
- Medicine information display.
- Responsive layout.

The JavaScript frontend communicates with the Spring Boot backend using the Fetch API.

4.3 API Integration

The frontend communicates with the backend through REST API endpoints.

The main endpoints include:

`/api/search`

`/api/count`

The search request is sent in the following form:

`/api/search?query=<search-query>`

The backend processes the query and returns medicine records in JSON format.

The frontend receives the JSON response and dynamically renders the medicine results.

4.4 Data Processing

The MedicineRepository reads the processed CSV dataset using Java file-handling classes.

The CSV parser handles fields containing quoted values and separates the medicine attributes into individual fields.

Invalid numeric records are skipped during loading so that a single malformed record does not stop the complete dataset from being processed.

The loaded dataset is stored as Java Medicine objects for efficient repeated searching.

5. RESULTS

5.1 Test Environment

The system was tested using the Java Spring Boot backend and browser-based frontend.

The processed medicine dataset contains approximately 242,381 records.

The backend successfully loads the complete dataset and exposes the search functionality through the REST API.

5.2 Functional Results

The implemented system successfully demonstrates:

- Loading of 242,381 medicine records.
- Medicine search through the web interface.
- Exact medicine-name searching.
- Approximate spelling-based searching.
- Composition-based searching.
- Manufacturer-based searching.
- Medicine type filtering.
- Result sorting.
- Favourite medicine functionality.
- API-based communication between frontend and backend.

The search system can also return relevant results for queries containing spelling variations.

5.3 Performance Observations

The system is designed to process a large medicine dataset containing hundreds of thousands of records.

KMP provides linear-time pattern matching after preprocessing, while Rabin-Karp uses rolling hashing for pattern detection.

Edit Distance provides approximate matching, while Cosine Similarity contributes to relevance ranking.

The combination of these algorithms allows the system to provide more useful results than a simple exact string search.

5.4 Screenshots and Analysis

The following screenshots can be included in this section:

- Pharma Search homepage.
 - Medicine search result page.
 - Search result for an exact medicine name.
 - Search result for a misspelled medicine query.
 - Medicine type filtering.
 - Result sorting.
 - Favourite medicines.
 - Backend terminal showing medicine dataset loading.
 - API search response.
 - Algorithm demonstration output.
-

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

Several challenges were encountered during development.

The first challenge was processing a large medicine dataset containing more than 240,000 records while maintaining acceptable search performance.

Another challenge was handling spelling variations because exact string matching alone cannot identify medicines when the user enters an incorrectly spelled query.

The project also required integrating multiple algorithms without breaking the existing search functionality.

Frontend-backend communication was another important challenge because the browser application depends on the Spring Boot backend running on port 8080.

6.2 Solutions Applied

The dataset was cleaned and stored in a processed CSV file before being loaded by the backend.

KMP, Rabin-Karp, Edit Distance, and Cosine Similarity were combined to provide different forms of matching.

A weighted relevance score was introduced to combine the results of multiple matching techniques.

The frontend communicates with the backend using REST API requests, allowing the search interface to display live results from the medicine dataset.

Additional algorithm implementations were organized into separate Java classes to keep the project modular and easier to demonstrate.

6.3 Limitations of the System

The current system uses CSV-based storage rather than a production database.

The search operation processes the medicine records in memory, which may require optimization for significantly larger datasets.

The relevance score is based on predefined weights and may not always represent human judgement perfectly.

The Cosine Similarity implementation is based on textual similarity and does not provide complete medical semantic understanding.

The system is intended primarily as an academic DSA project and is not designed to provide medical diagnosis or treatment recommendations.

The backend must also be running for the browser frontend to retrieve live search results.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Pharma Search – An Intelligent Medicine Search and Recommendation System demonstrates how Data Structures and Algorithms can be applied to a practical medicine-search problem.

The project combines a large medicine dataset, Java backend, Spring Boot REST API, browser-based frontend, and multiple algorithmic techniques into a single application.

KMP, Rabin-Karp, Z-Algorithm, Edit Distance, and Cosine Similarity demonstrate different approaches to string matching and similarity analysis. The combined ranking mechanism allows the system to provide more relevant search results for both exact and approximate queries.

The project also includes implementations corresponding to the broader DSA-3 course outcomes, including network flow, approximation, randomized algorithms, and parallel algorithms.

7.2 Future Work

Future improvements may include:

- Replacing CSV storage with a database for production-scale deployment.
 - Improving the relevance-ranking model.
 - Adding more advanced semantic search techniques.
 - Improving medicine-name and composition normalization.
 - Adding pagination for very large search-result sets.
 - Deploying the Spring Boot backend permanently using a cloud platform.
 - Adding detailed medicine information pages.
 - Improving the recommendation component.
 - Adding automated performance benchmarking.
 - Extending the frontend with additional search and filtering capabilities.
-

7.3 Skills Learned During Development

The project provided practical experience in:

- Java programming.
- Spring Boot development.
- REST API development.

- HTML, CSS, and JavaScript frontend development.
 - CSV and file handling.
 - KMP string matching.
 - Rabin-Karp rolling hash.
 - Z-Algorithm.
 - Dynamic programming and Edit Distance.
 - Cosine Similarity.
 - Algorithmic result ranking.
 - Network-flow algorithms.
 - Approximation algorithms.
 - Randomized algorithms.
 - Parallel algorithms.
 - Git and GitHub version control.
 - Debugging and integration testing.
 - Connecting algorithmic concepts with a practical software application.
-

REFERENCES

- [1] Oracle, *Java Platform, Standard Edition Documentation*, Java documentation.
- [2] J. H. Morris and V. R. Pratt, “A Linear Pattern-Matching Algorithm,” 1970.
- [3] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, 1977.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
- [5] R. Wagner and M. Fischer, “The String-to-String Correction Problem,” *Journal of the ACM*, 1974.
- [6] Documentation for Spring Boot and related Java web technologies.

